

Week 2- Algorithm Correctness and Mathematical Induction

Condensed Study Notes

Generated: 2026-05-25 10:18 UTC

Week 2- Algorithm Correctness and Mathematical Induction

This section focuses on proving that algorithms work as intended (correctness) using a powerful mathematical tool: mathematical induction.

Algorithm Correctness: Ensuring an algorithm produces the correct output for all valid inputs.

Mathematical Induction: A proof technique used to establish that a statement is true for all natural numbers (or a subset starting from a base case). It involves two main steps:

1. **Base Case:** Prove the statement is true for the smallest value (e.g., $n=0$ or $n=1$).
2. **Inductive Step:** Assume the statement is true for an arbitrary value ' k ' (the inductive hypothesis) and then prove it must also be true for the next value, ' $k+1$ '.

By successfully completing these two steps, mathematical induction establishes the truth of the statement for an infinite sequence of numbers.

Objectives

This section focuses on developing your ability to verify algorithm correctness. You will learn to:

- Understand what algorithm correctness means.
- Apply loop invariants to prove the correctness of algorithms that use loops.
- See how mathematical induction, previously covered, is used for proving correctness.

What is algorithm correctness

Algorithm correctness means an algorithm produces the expected results for all possible inputs.

It's also crucial that the algorithm terminates (stops running). Therefore, correctness has two aspects:

- **Partial Correctness:** The algorithm produces the correct output if it terminates.
- **Total Correctness:** The algorithm produces the correct output AND it is guaranteed to terminate.

Partial correctness

Partial correctness guarantees that if an algorithm terminates, it produces the correct output. It does not, however, guarantee that the algorithm will terminate at all.

This is crucial for algorithms where logical accuracy of results is paramount, even if they are designed to run indefinitely. For instance, monitoring systems must be partially correct for their ongoing output to be

reliable, despite not having a defined endpoint.

Example: A search algorithm that correctly finds an element if it terminates, but might loop forever.

Total correctness

Total correctness means an algorithm not only produces the correct result but also finishes within a finite number of steps. This is the ideal scenario we aim for in well-defined algorithms, as an algorithm that doesn't complete is typically of limited use.

Example: A program that sums numbers from 1 to n has total correctness because it always provides the correct sum and terminates for any valid input n .

Significance of algorithm correctness

Algorithm correctness is paramount in various critical fields, ensuring dependable and predictable software.

Key areas where correctness is vital:

- **Safety-critical systems:** For example, aviation control algorithms must maintain stable flight paths.
- **Finance:** Banking algorithms for calculations like interest must be accurate to prevent financial errors.
- **Healthcare:** Ensuring correct operation in medical software.

Correct algorithms avoid errors in these sensitive domains, making software reliable and safe.

Introduction to Mathematical Induction

Mathematical induction is a proof technique used to demonstrate that a statement or formula is true for all natural numbers.

It involves two key steps:

- **Base Case:** Prove the statement is true for the first natural number (typically $n=1$).
- **Inductive Step:** Assume the statement is true for an arbitrary natural number ' k ' (this is the inductive hypothesis), and then prove that it must also be true for the next natural number, ' $k+1$ '.

Examples of mathematical induction

This section demonstrates how to use mathematical induction to prove a formula.

Example: Prove that the sum of the first n positive integers is $S = (n(n+1))/2$.

To prove this using induction, we follow these steps:

1. **Inductive Hypothesis:** Assume the formula holds for $n = k$. This means we assume $S = (k(k+1))/2$ is true.

2. **Inductive Step:** Show that if the formula holds for $n = k$, it must also hold for $n = k+1$. We need to show that $S = ((k+1)((k+1)+1))/2$.

By demonstrating these two steps, we can confirm the formula is true for all positive integers n .

Strong Induction

Strong induction, also called the second principle of mathematical induction, is a proof technique for establishing a property for all natural numbers.

Unlike simple induction, strong induction assumes the property holds for *all* preceding natural numbers up to a given point, not just the immediately preceding one.

How it works

Strong induction requires proving a property for a base case (initial value, e.g., $n=0$ or $n=1$) and then, in the inductive step, assuming the property holds for all values up to n ($P(1)$, $P(2)$, ..., $P(n)$) to prove it for $n+1$.

This method is ideal for recursive algorithms because it validates correctness as solutions build upon smaller instances of themselves.

Process

To prove an algorithm's correctness, follow these steps:

1. **Define the Statement:** Clearly articulate what the algorithm is designed to achieve.
2. **Base Case:** Verify the algorithm's correct behavior for the simplest possible input.
3. **Inductive Step:** Demonstrate that if the algorithm functions correctly for an input of size ' k ', it will also function correctly for an input of size ' $k+1$ '. This step is often illustrated using a recursive function, like calculating the factorial of ' n '.

Proving Algorithm Correctness with Induction

This section details how mathematical induction, specifically strong induction, can be applied to prove algorithm correctness.

Core idea: To prove an algorithm is correct for all inputs, we can use induction. This involves showing it works for the simplest cases (base case) and then proving that if it works for *any* set of inputs, it also works for the next larger set of inputs (inductive step).

Strong induction is particularly useful here because it allows us to assume the property holds for all preceding cases, not just the immediately preceding one. This aligns well with how many algorithms are structured recursively.

Process outline:

1. Define the specific correctness property to prove.
2. Establish the base case: Prove the property holds for the smallest/simplest input(s).
3. Prove the inductive step: Assume the property holds for all inputs up to some value 'k'. Then, show that it must also hold for input 'k+1'.

Introduction to Loop Invariants

A loop invariant is a condition that remains true both before the loop starts and after each complete iteration. Its purpose is to prove the correctness of iterative algorithms by demonstrating that the loop consistently moves closer to achieving its goal.

Characteristics of a Good Loop Invariant

A useful loop invariant should satisfy three core properties to aid in proving algorithm correctness:

- **Initialisation:** The invariant must be true before the loop begins.
- **Maintenance:** The invariant must remain true after each iteration of the loop.
- **Termination:** When the loop finishes, the invariant, combined with the loop's termination condition, must help prove that the algorithm has reached its intended goal.

Additionally, a good loop invariant is clear and directly related to the algorithm's overall objective. It's typically placed at the start of the loop. A well-chosen invariant is valuable for designing, testing, and modifying code, serving as documentation and a foundation for correctness proofs.

Application of induction to iterative algorithms: Example 1

This section applies induction to prove the correctness of the iterative algorithm POW2(n), which aims to return 2^n .

We observe the pattern of program variables through loop iterations.

Let's assume the loop has run for 't' iterations. At this point, the variable 'power' holds 2^k , where k is the number of iterations performed. The loop invariant states that $\text{power} = 2^t$ after t iterations.

We also track an iteration counter, say 'i'. If after 't' iterations, $i = n$ and $i = t$ (as i increments with each iteration), then $t = n$.

Since i increases by 1 in each step ($i_{k+1} = i_k + 1$), i will eventually reach n, causing the loop to terminate.

POW2(n) terminates with the 'power' variable correctly holding 2^n .

Correctness proof summary for POW2(n):

- **Goal:** Prove POW2(n) returns 2^n for all natural numbers.
- **Loop Invariant:** $\text{power} = 2^t$ after t iterations.

- **Termination:** The loop terminates when the iteration counter i equals n .
- **Conclusion:** Upon termination (when $t = n$), the loop invariant guarantees $\text{power} = 2^n$.

Application of induction to iterative algorithms: Example 2

This section applies induction to prove the correctness of an iterative algorithm, EVENSUM(n).

To find the loop invariant, we observe the pattern of program variables within the loop over each iteration.

For EVENSUM(n), we consider the state at iteration k (denoted by subscript k).

Initial state ($k=0$): $i0 = 1$ and $even0 = 0$.

Inductive step: The proof uses induction on k .

Induction hypothesis: Assume for $k \geq 0$, the following holds: $ik = k + 1$ and $evenk = k \sum_{i=1} 2^i - 2$.

We aim to prove that for all natural numbers $k \geq 0$, $ik = k + 1$ and $evenk = k \sum_{i=1} 2^i - 2$.

Proving EVENSUM(n) Termination

This section focuses on demonstrating that the EVENSUM(n) algorithm terminates and produces the correct result.

Key points:

- The proof relies on the established loop invariant, which guarantees conditions hold true before and after each loop iteration.
- The algorithm is shown to terminate when the loop counter i eventually reaches $n + 1$.
- This termination condition is guaranteed to be met after a finite number of iterations, denoted as t .
- The proof of termination and the correct result is intrinsically linked to the initialisation and maintenance phases of the loop invariant, essentially completing the inductive proof.

Inductive Step

The inductive step assumes the algorithm's correctness for a range of values up to a certain point, and then proves it holds for the next value.

For the POW2(k) algorithm, the induction hypothesis states that POW2(k) correctly returns 2^k for all values from 0 up to and including k , where k is greater than or equal to 2.

Strong Induction on Recursive Algorithms

This section applies strong induction to prove the correctness of recursive algorithms, using the Recursive POW2(n) function as an example.

Goal: Prove that for all $n \geq 0$, POW2(n) returns 2^n .

Base Case (n=0):

- POW2(0) returns 1.
- This matches $2^0 = 1$, so the base case holds.

Strong Induction: Recursive EVENSUM Example

This section applies strong induction to prove the correctness of the recursive EVENSUM(n) algorithm.

Claim: For all $n \geq 1$, EVENSUM(n) returns the sum of the first n even numbers, represented as $\sum_{i=1}^n 2i - 2$.

Base Cases:

- For $n = 1$, EVENSUM(1) returns 0 (as claimed).
- For $n = 2$, EVENSUM(2) returns 2 (as claimed).

Inductive Step:

Induction Hypothesis: Assume that $k \geq 2$ and EVENSUM(k) correctly returns $\sum_{i=1}^k 2i - 2$ for all numbers from 0 up to k.

Proof for EVENSUM(k+1): We need to show that EVENSUM(k+1) also returns the correct sum.

- What EVENSUM(k+1) actually returns is $\sum_{i=1}^{k+1} 2i - 2$.

Conclusion: Therefore, for all $n \geq 1$, EVENSUM(n) correctly returns $\sum_{i=1}^n 2i - 2$.

Activities

Practice applying algorithm correctness techniques.

- **Activity 1: Proving Correctness with Induction**

- Focus: Practice proving the correctness of recursive algorithms.
- Example: Prove the correctness of a recursive function that calculates the ■■■ factorial.

- **Activity 2: Identifying Loop Invariants**

- Focus: Practice identifying loop invariants in simple iterative algorithms.
- Example: Verify the loop invariant for an algorithm that finds the maximum element in an array.

Summary

This section summarises the key techniques and concepts covered for proving algorithm correctness:

- **Algorithm Correctness:** Defined and its importance stressed, ensuring algorithms produce correct outputs and terminate.
- **Proof Techniques:** Mathematical induction and loop invariants were introduced and practiced as tools to verify algorithm correctness.

- **Practical Application:** These concepts were applied to simple algorithms, demonstrating their utility in ensuring reliability and predictability in systems.

Further tasks

To deepen your understanding of algorithm correctness:

- **Practice Problems:** Actively seek out and solve problems that require proving algorithm correctness using mathematical induction and loop invariants. Explore online resources and personal research for these exercises.
- **Real-World Application:** Consider and identify other scenarios in everyday life or professional settings where ensuring algorithm correctness is critically important.